Loki III: Reverse Engineering Structural Netlists without a Language Model
How far do provenance, a Bayesian L1 and a graph L2 get on their own?

Peter Jamieson
Electrical and Computer Engineering, Miami University
Oxford, OH, USA
jamiespa@miamioh.edu

***Abstract*—Gate-Breaker showed that a language model can recover functional RTL from a gate-level netlist, and it ended by naming the graph techniques its model needed: clustering to name buses, a classification of signals as datapath or control, and partitioning. Loki III asks the complementary question: how much structure can be recovered *without* a language model, from provenance, a Bayesian hypothesis flow and graph analysis alone? This paper describes the benchmarks built to answer that question and reports the recovery statistics under the Gate-Breaker metrics. The benchmarks are 68 synthesizable micro designs and the 156 VerilogEval references (150 build on three floors), compiled by a provenance-tracked front end into named, labelled, anonymised netlist datasets; a renamed-instance protocol then proves the pipeline naming-neutral across 105,646 renamed identifiers. On the L1 hypothesis flow, the structures that are already reverse engineered by the synthesis flow recover almost perfectly (hard adders 67 of 67, hard multipliers 30 of 30), while the structures the flow erased recover weakly (macro-F1 0.354 on the first floor, 0.108 on the second, calibrated). The L2 graph stage beats a 1-WL baseline on real micros once node, graphlet, WL and NPN features combine (macro-F1 0.4241 against 0.2717) and its clusters carry the designer's hierarchy (Louvain modularity 0.4220). We state the honest limits: no language model step exists, the recovered structure is structural rather than functional, and the paper's own determinism defects are reported.**

***Index Terms*—reverse engineering, netlist, provenance, Bayesian inference, graph neural networks, FPGA CAD**

I. Introduction

Gate-Breaker [1] takes a gate-level netlist with obfuscated wire names, formats it into a four-step prompt (interface, decomposition, function, generation), asks a language model to write RTL, and checks the result by simulation. On 156 VerilogEval problems [2] its best model recovered functional RTL for 75.1% of the runs, and 53.4% on the hardest quartile. The paper's own conclusion is the interesting part: external graph tools should help the model – clustering to name buses, a classification of signals as datapath or control, and partitioning of the netlist into smaller parts.

Loki III (https://github.com/drpaj12/loki) is the Yggdrasil III suite's reverse-engineering tool, and it targets exactly those three items – with no language model step at all. Its pipeline is L0, a provenance ground-truth dataset made from the benchmarks; L1, a Bayesian hypothesis flow whose pattern detectors propose structures and whose proofs confirm them; and L2, graph analysis (clustering, structural features, graph neural networks) over the same netlists. The research question this paper answers is: *how good are the L0, L1 and L2 experiments, without more intelligence, at recovering structure?* The answer matters for two audiences. For the Gate-Breaker line, it prices the graph side of the future work the paper names. For reverse engineering, it measures what a provenance-tracked front end and honest statistical machinery can do before any model is added.

The contributions are: (1) a benchmark method – provenance-tracked, labelled, anonymised netlist datasets with a renamed-instance protocol that proves naming neutrality, so the pipeline can be evaluated on designs whose names carry nothing; (2) the L1 and L2 recovery statistics under the Gate-Breaker metrics and under Loki's own (precision, recall, calibration, clustering quality, GNN against a 1-WL baseline); (3) the honest limits, including two determinism defects the experiments found.

II. Background

A. The provenance problem and Odin III

Synthesis modifies the designer's functional intention: flattening removes hierarchy, optimization merges operators, and mapping covers logic with cells that carry no source names. Linking each netlist object back to the HDL that produced it is the provenance problem [3]. Odin III, the provenance-tracked front end under Loki, keeps that link: every object of its one IR records its source location, and the link survives elaboration, optimization, partial mapping and the linked ABC flow. Loki stands on that link: its L0 datasets carry the answer key the synthesis tool kept, which is what makes the recovery statistics in this paper possible at all.

B. Gate-Breaker as the method baseline

Gate-Breaker measures recovery by functional correctness and by the similarity of the recovered and reference RTL:

Success Rate (the fraction of K samples that simulate correctly), Success@k (the fraction of designs solved within the first k samples), AST similarity (mean subtree Jaccard over normalized identifiers, depth $d = 3$) and CFG similarity (a normalized shortest-path node kernel over control-flow graphs). Its complexity score weights sequential blocks, generates, loops, instantiations, always blocks, buses, multiplexers, cases, arithmetic operators, assignments and concatenations into quartiles Q1 (simple combinational) through Q4 (complex sequential). Loki reports its own statistics on the same axes where they apply – recovery rate per structure kind, calibration error, clustering quality against the known hierarchy, and GNN accuracy against a 1-WL baseline – and the Gate-Breaker metrics on the replication runs.

III. Methodology: floors, pipeline and evaluation

A. Floors: the abstraction level of the netlist

A *floor* is the abstraction level and technology mapping at which an Odin III design is cut and written out as the L0 netlist, the object the reconstruction receives. Every floor starts from the same elaborated design, the common front `proc; fsm; opt; flatten; opt -full; wreduce` (`FRONT` in `loki/odin.py`), and then maps it with a floor-specific script (`floor_script` in the same file). Table I lists the floors. Four exist: `f1-k6` and `f1-earch` (one floor per FPGA architecture file; the directory name is `f1-` plus the architecture's lower-case prefix), `f2`, and `f3`.

What a floor's netlist contains decides what is left to reconstruct. At `f1` the flow's `hardmap` step has already recognised adders, multipliers and memories, so they appear as typed hard cells; the construct they implement is part of the netlist, and the labels still name it. At `f2` the same logic is only LUT truth tables: an adder is a carry chain of LUTs that has to be found again. At `f3` the logic is a network of two-input gates, the form Gate-Breaker's language model receives. The `f2` output does not depend on the architecture file, and `f3` reads none, so their files are byte-identical across architectures.

Loki uses several floors because what a synthesis flow erases depends on the mapping, and recovery has to be measured against each. `f1` keeps the structures the flow already reverse engineered, which is what lets the `hard` detector of L1 read them from Odin III's `unmap` facts; `f2` removes that crutch with the same source and the same answer key; `f3` makes the input comparable to Gate-Breaker's and is the floor on which the AST and CFG similarities are computed (Section V-C). Within one design the floors share one source of truth, the provenance spans of the same elaborated IR, so a difference between floors is a difference of the mapping.

B. L0: provenance ground-truth datasets

The first benchmark set is the 68-design micro suite (counters, registers, arithmetic, ALUs, muxes, comparators, small FSMs) that Odin III already compiles and checks against Yosys; the second, the 156 VerilogEval references, is described in Section IV. L0 compiles each design on every floor of Table I. Every netlist object carries its provenance (source file and line), a neutral `n`k name from one shared counter, and a label: the structure kind (`construct_type`) is the answer key in `labels.csv`, and the netlist-visible hard-block kind (`cell_type`) travels in the graph labels, so structures that are already reverse engineered stay visible to the reconstruction. `loki check` validates every dataset: 68 designs, 269 floors, 100% labelled rows, zero errors. The two hard-block designs the v1.0 front end refused now compile on the v1.1.0 SystemVerilog front end, so the set is complete.

Renamed instances. A separate protocol asks the naming question directly: rename every identifier of every netlist (105,646 identifiers across 269 netlists) through a shared permutation, run the pipeline on the renamed floors, and inverse-map every token back. The verifier reports token-for-token equality with the original, zero failures, and the base and renamed score digests are byte-identical (sha256 over all run scores, 204 of 204 pairs) – the pipeline is naming-neutral, so nothing it recovers is an artefact of a name.

C. L1: the Bayesian hypothesis flow

L1 is a round-based hypothesis flow over one netlist, with no language model. It reads a floor netlist into a fresh Odin III design (level 0, never edited) in which every cell is a function, and keeps a working copy that grows one level of hierarchy per committing round.

Hypotheses and detectors. A hypothesis is a claim that a set of level-0 cells is a structure of some kind, for example: a flip-flop with enable or synchronous reset (`ff_en`, `ff_srst`), a `register`, a `bus` of isomorphic slices, an `adder`, `subtractor`, `comparator` or constant comparison (`eq_const`), a `mux` or `mux_tree`, a `counter`, `shift_reg`, `lfsr` or `fsm`, a datapath or control `unit`, a per-cell `role`, or a hard block (`hard_add`, `hard_mul`, `hard_mem`). Nine detector plugins (arith, bus, ff_pattern, fsm, hard, mux, role, seqfn, unit) propose them. A detector reports features as *bin labels* (a width of 5 to 8, whether an enable function is exact, the NPN class of a cell) and never a likelihood; the likelihoods come from calibration.

Bayesian inference. Each hypothesis h of kind k from detector d carries a log-odds $L(h)$, the logarithm of the odds that it is a real structure, and $p(h) = 1/(1 + e^{-L(h)})$ is its probability. L is a naive-Bayes sum: a prior plus evidence items, each the log-likelihood ratio (LLR) of an observation,

$$L(h) = \pi_{d,k} + \mathrm{clip}_{\pm 6.91}\Big(\sum_f w_{\phi(f)}\, \lambda_{k,f}(b_f) + \min\big(1.10\, n_{\mathrm{conf}},\, 3.30\big) - 2.30\, n_{\mathrm{ref}} \Big) + e_h, \quad (1)$$

where $\pi_{d,k}$ is the prior log-odds of that detector and kind; $\lambda_{k,f}(b_f)$ is the LLR of feature f in bin b_f; $w_{\phi(f)}$ weights the feature's family (structural, WL colour or NPN class); n_{conf} and n_{ref} are the numbers of confirmed (or committed) and refuted child hypotheses (a bus slice, a register's flip-flops), which raise and lower their parent; and e_h collects fixed records (a sampled check adds $\ln 100 = 4.61$). The clip

TABLE I
THE FLOORS OF L0. ALL FLOORS RUN THE COMMON FRONT FIRST; THE MAPPING SCRIPT IS THE SECOND PART OF `FLOOR_SCRIPT`. NODE KINDS ARE THOSE OF `GRAPH.JSON` (`PI`, `PO`, `CONST`, `LUT`, `FF`, `HARD`, `GATE`).

Floor	Reads	Mapping script	Netlist contents
`f1-k6`	`read_arch k6_frac_N10_ frac_chain_mem32K_40nm`	`hardmap; lower; abc --lut 6`	LUT6, flip-flops, and hard adder, multiplier and RAM cells that keep their Odin III type
`f1-earch`	`read_arch` of the EArch file	same as `f1-k6`	same kinds, EArch hard blocks
`f2`	`read_arch` (cell types only)	`lower; abc --lut 6`	LUT6 and flip-flops only; arithmetic is soft, so no `hard` node exists
`f3`	`read_techlib lib/cmos6.o3lib`	`lower; abc --gates cmos6; dfflibmap cmos6`	library gates (BUF, INV, NAND2, NOR2) and DFF flip-flops

at $\pm \ln 1000 = 6.91$ limits the double counting that a naive-Bayes sum of correlated features would otherwise allow. A proof sets $L(h) \geq \ln 9999 = 9.21$ (the odds become 9999 to 1), and a counterexample sets $L(h) = -\infty$.

All three tables are *calibrated* from the training split of L0 (`loki calibrate`). The flow runs with default tables on every training design; each hypothesis is labelled true when it matches an L0 construct (Section III-E). The prior is the Laplace-smoothed precision of the detector, $\pi_{d,k} = \operatorname{logit} \frac{tp+1}{n+2}$ over n proposals of which tp are true. For example, the `f3` comparator detector made 31,036 proposals of which 2 were true, so $\pi = \ln \frac{3}{31035} = -9.24$; the committed value is -9.244259 (`experiments/f3-recovery/calib/f3.json`). A bin's LLR is $\lambda = \ln \frac{(pos+1)/(POS+B)}{(neg+1)/(NEG+B)}$, with pos and neg the true and false proposals in the bin, POS and NEG the totals of the kind, and B the number of bins. The family weights w are the grid values in $\{0, 0.1, \ldots, 1\}$ that maximise the Bernoulli log-likelihood of the labels, fitted by three sweeps of coordinate ascent on out-of-fold rows (two folds, split by design). Without calibration (`--calib none`) the priors are 0 and the LLRs are the defaults of `bayes.py`. The reliability check is the expected calibration error over ten equal-width bins of the predicted probability, $\mathrm{ECE} = \sum_b \frac{n_b}{N} |\bar{p}_b - r_b|$ with r_b the observed match rate.

Proof engines. A hypothesis with a functional claim ("these 9 LUTs compute $A + B$ at width 5") can be proved. Three engines prove claims: `cone_check` first when the claim has a template, the truth table otherwise or when the cone cannot be bound to the template, and transfer after an exact proof. (i) `cone_check`, an Odin III pass, proves that the claim's cone of level-0 cells is equivalent to a template word cell (for example `$add`, `$sub`, `$eq`, `$mux`) with its ports bound to the cone's boundary nets, under a timeout; the reference functions of the claims (`claims.py`) cover addition, subtraction, the six comparisons, constant comparison, 2:1 and N:1 multiplexers, counters, shift registers and LFSRs. (ii) The *truth-table* prover enumerates the claim's inputs when they number at most 16, an exact proof or a counterexample, and otherwise simulates 4096 seeded random vectors, which can refute a claim but only raises it by $\ln 100$. (iii) *Proof transfer* groups open hypotheses by operation, parameters and the 1-WL colour of their output drivers; when one member is proved exactly, the others are tried by a lockstep `cone_match` of the two cones, and one transferred member per group is proved for real as a spot check, revoking the transfers of the group if it fails. A sequential claim is proved as a next-state function (state in, next state out); an FSM gets no proof.

Deciding the structure. Hypotheses that claim the same cells compete. Two hypotheses *conflict* when both kinds are exclusive, their cell sets intersect, and neither is a strict subset of the other (a strict subset is nesting: the smaller becomes a child of the larger). For each connected component C of the conflict graph the competitive probability is

$$p_{\mathrm{comp}}(h) = \frac{e^{L(h)}}{1 + \sum_{j \in C} e^{L(j)}}, \tag{2}$$

where the "1" is the option that none of them is real; a hypothesis without conflicts has $p_{\mathrm{comp}} = p$. Two confirmed hypotheses with equal cells are two true forms of the same logic and count as one member of C; the one whose kind ranks higher (a counter above an adder, for example) wins at commit and the other is kept as an *alternative*. A proof is spent on hypotheses with $p_{\mathrm{comp}} \geq 0.2$, most probable first, at most 200 per round; a hypothesis below 0.01 is rejected, and re-enters when later evidence lifts it. A hypothesis is *selected for commit* when $p_{\mathrm{comp}} \geq 0.9$, every child that could commit is already committed, and it does not conflict with a committed piece. Selection is greedy in the order (p_{comp} descending, kind rank, cell count descending, id), and the losers of a conflict become rejected or alternative.

Commit and the final structure. A selected hypothesis is rebuilt on a copy of the working design: a proved word-level claim is *raised* to a single word cell (re-checked by `cone_check` first when the proof was a `cone_check` one) and every selected piece is wrapped by `submod` in a module named for its kind, whose instance is tagged with the hypothesis id, kind, confidence p_{comp}, proof status and the recovered function. If `raise` or `submod` fails, the offending hypothesis is rejected with a record of the error and the commit is retried once without it (`reject_failed`). The structure that L1 reports is therefore exactly the set of committed pieces in `design.v`; the alternatives, rejected and open hypotheses stay in `overlay.json` with their evidence. Algorithm 1 summarises the loop. It stops when a round commits nothing, proposes nothing new and changes no log-odds by more than 10^{-6} (a fixed point), after eight rounds, or at the time budget, then flattens and lowers the final design and checks it against

Algorithm 1 The L1 round loop (`loki/re/loop.py`).

```
level_0 ← read netlist as functions; work ← level_0
if floor f1 with hard cells then run unmap on work
for r = 1 to 8 do
    propose: every detector adds hypotheses over the view of work
    score: L(h) by Eq. 1; compete: p_comp by Eq. 2
    prove claims with p_comp ≥ 0.2: cone_check, truth table, transfer
    re-score and compete; reject p_comp < 0.01
    select p_comp ≥ 0.9 (children first); commit by raise and submod
    if nothing committed, nothing new, |ΔL| ≤ 10^−6 then
        break
    end if
end for
write design.v, overlay.json, report; equiv-check against the input
```

the input with `equiv-check`; a result of `differs` is a bug and the run exits with an error.

The `hard` detector reads the `unmap` pass's facts: a hard adder, multiplier or memory that the flow already reverse engineered is recovered directly, with the decline reasons the pass recorded (carry-in chains and the like) kept as open hypotheses. The `hard` detector does not run on `f2` or `f3`.

D. L2: graph analysis

L2 reads the L0 `graph.json` files and nothing else: never `key.json` and never the BLIF. It has three parts.

Structural features. Each node gets numeric features (fan-in, fan-out, logic level, LUT width, hard-port count), its kind and, on hard cells, its `cell_type`, plus three families: colours of 1-WL colour refinement (a hash of the node's label and the multiset of its neighbours' colours, per round, so that equal colours mean equal unfolded neighbourhoods), the NPN class of a LUT's function (invariant under permutation and negation of the inputs and the output), and the ORCA orbit counts of the 4-node graphlets around the node. Before graphlets are counted, the global signals (clock, reset, constants, global enables) are classified by role and their edges removed, so that a clock does not connect everything.

Clustering. Four methods partition a netlist graph into candidate blocks: k-means on the feature space ($k = \mathrm{clip}(\mathrm{round}\sqrt{N/2}, 2, 200)$), affinity propagation on a sample of at most 3000 nodes with the remaining nodes assigned to the nearest exemplar, spectral clustering (a spectral embedding of the largest connected component, then k-means), and Louvain community detection (resolution 1, no k). The first two use node features; the last two use the graph structure. A partition is scored by its *modularity* on the undirected simple projection of the graph and by its *NMI* (and ARI) against the known `module_path` and `construct_type` labels over the LUT, flip-flop, hard and gate nodes. These two numbers are a quality report, never an input to a method: modularity says whether the clusters are dense subgraphs, NMI whether they are the designer's modules.

GNN against 1-WL. A node classifier predicts each node's `construct_type`: a two-layer GraphSAGE (GCN as the variant) of hidden width 64 with ReLU and dropout 0.2 over the graph and its reverse edges, trained with Adam (learning rate 0.01, weight decay $5 \cdot 10^{-4}$) for a fixed 100 epochs, full-batch, with class weights $1/\sqrt{\mathrm{count}}$. The split is L0's, by design, with the class vocabulary taken from the training designs only (rarer values, and values first seen in a test design, are one class `__other__`); there is no validation set and no early stopping, so nothing is tuned on a test design. Loss and metrics cover only the LUT, flip-flop, hard and gate nodes with a known target. The question is whether message passing earns anything over what the colour refinement already knows, because a message-passing GNN is at most as discriminating as 1-WL [4]. So the 1-WL baseline predicts, for each node, the most common training class of its WL colour (backing off from round 4 to round 0 when a colour has fewer than five training nodes), and the GNN has beaten it only if its test macro-F1 is higher. Two further baselines are reported on the same nodes, the majority class and a logistic regression on the features alone. The task is that of GNN-RE [5], which maps each gate of a flat netlist to the module it belongs to; Loki's model is a baseline for the same task, not a re-implementation, and the numbers are not compared with that paper's.

E. Evaluation protocol: how L1 and L2 are scored

Gate-Breaker evaluates a stochastic model: for each design the model is sampled K times, each sample's RTL is extracted and simulated against the VerilogEval testbench, and the results are aggregated per complexity quartile as the success rate (the fraction of the K samples that pass), Success@k (the fraction of designs with a passing sample among the first k), and the AST and CFG similarity of the recovered RTL to the reference. Loki's L1 and L2 are deterministic and their output is a structure, not an RTL text, so the same four questions are asked in different terms. Table II sets the two side by side.

L1, step by step.

1) *Split.* L0 assigns each design to train or test by a hash of the design name ($\mathrm{sha256}(\text{seed:design}) \bmod 100 < 80$ is train), so the split is fixed by the name and independent of the run.
2) *Calibrate* on the training designs of one floor (`loki calibrate`): the priors, bin LLRs, family weights and reliability curve of Section III-C.
3) *Run* `loki re` on every test netlist, calibrated and, as a diagnostic, with `--calib none`. The run never reads the answer key.
4) *Ground truth.* From `words.json`, the L0 word cells give ground-truth instances (type, F_g) with F_g the set of final cells of the word; counters, mux trees and FSMs are derived from the word connectivity.
5) *Count and match.* A hypothesis *counts* when it is committed or its $p_{\mathrm{comp}} \geq 0.5$ (`--score-p`); it *matches* a ground-truth instance of an accepted type when the Jaccard of their cell sets is at least 0.5, greedily one-to-one by Jaccard. Precision is matched over counted,

TABLE II
THE GATE-BREAKER EVALUATION AND THE LOKI EVALUATION, STEP BY STEP.

Step	Gate-Breaker	Loki
Input	cmos6 gate netlist of a VerilogEval reference, obfuscated wire names	a floor netlist (`f1`, `f2`, `f3`) with names `n`k; for the neutrality check a renamed twin
Producer	a language model, K samples per design	the L1 flow, one deterministic run per design (fixed seed); no sampling
Correctness	simulate the extracted RTL against the testbench: success rate, Success@k	`equiv-check` of the flattened final design against the input: `final_equiv` (formal; skipped on `f1`, whose hard cells are not checked)
Match to the truth	AST and CFG similarity to the reference RTL	each counted hypothesis against the L0 answer key: Jaccard ≥ 0.5, one-to-one; precision, recall, F1, average precision per kind and floor; hierarchy similarity; AST and CFG of `design.v` against the reference on `f3`
Aggregate	per complexity quartile	micro-averaged per kind and floor over the test split; AST and CFG per quartile of `f3` node count

recall matched over ground truth, and average precision sweeps the threshold from 0.1 to 0.9.

6) *Buses, roles and hierarchy.* A bus matches a ground-truth bus (nodes sharing a module path and bus name, at least two bits) at the same Jaccard; its bit order is scored exactly and up to reversal. Roles are scored as precision and recall of the predicted control and datapath cells. The hierarchy similarity compares the clusters of the module paths to the committed pieces with the Dice score $F(g,h) = 2|g \cap h|/(|g| + |h|)$, $\mathrm{HS}_{gt} = \sum_g |g| \max_h F(g,h) / \sum_g |g|$ and its converse HS_{l1}, and reports their harmonic mean.
7) *Aggregate.* `loki score` writes one `score.json` per run and `loki score-all` the floor-by-kind tables, the reliability curve of p_{comp} with its ECE, and, with `--compare`, the ablation against `--features none`.
8) *Naming neutrality.* The renamed twin of the dataset is run and scored against the original-named key; the base and renamed `score.json` must be byte-identical.

Of the Gate-Breaker metrics, the AST and CFG similarities are used on `f3` as in the paper: AST similarity is the mean, over the subtrees of the reference, of the best Jaccard similarity to a subtree of the candidate (depth 3, node kinds normalised); CFG similarity is the node kernel over shortest-path histograms of the control-flow graphs. They are computed twice per design, for the gate netlist (L0-stop) and for `design.v` (L1-stop), and the L1 delta is their difference. The success rate and Success@k need samples; a deterministic flow has one attempt per design, so they are reported only for the language-model replication (Section V-E), and formal `final_equiv` plays the role of the testbench.

L2, step by step. (1) The same split applies. (2) `loki analyze cluster` writes a partition per method and design, with its modularity and NMI/ARI against `module_path` and `construct_type`; the table reports the mean over designs. (3) `loki analyze gnn` fits the vocabulary and the model on the training designs and writes `metrics.json`: accuracy and macro-F1 on the training and test nodes for the GNN, the majority class, the feature-only logistic regression and the 1-WL baseline, and `beats_wl`. (4) The neutrality check re-runs both on a permuted copy of every identifier.

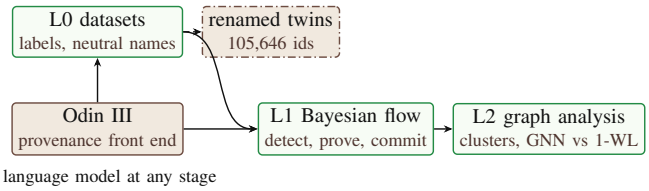


Fig. 1. The Loki III pipeline. L0 compiles the benchmarks through Odin III into provenance-tracked, labelled, anonymised datasets (and their renamed twins). L1 proposes and proves structures in a Bayesian flow that commits into the same IR. L2 reads the graphs back and scores clustering and a GNN against a 1-WL baseline. No stage has a language model.

F. One design through the pipeline

The steps below follow one design, the 4-bit comparator micro `bm_DL_4_bit_comparator` (three comparison outputs), from source to score. Each step names the file or code that defines it.

1) *HDL to Odin III with provenance.* `read_verilog` and the common front elaborate the design. Every IR object keeps its source spans (`Node.sources()`, the only provenance the binding exposes). Before mapping, Loki snapshots every word-level cell (type, parameters, spans) and every instance site.
2) *Map to the floor.* `lower` keeps each word cell's exact span on every gate it creates; `abc -lut 6` gives each LUT the union of the spans of its cone. A LUT that merges the three comparators therefore lists all three spans.
3) *Join and label.* A final cell is matched to a word cell when the word's span set is a subset of the cell's. The primary match gives `construct_type` (here `$eq`, `$gt`, ...); a cell with several matches is flagged (`n_construct_cells` above one) and a cell with none stays `unknown`: a label is an answer key, never a guess. The module path comes from the instance spans, bus name and bit index from net names and aliases (`AeqB[0]`), and the datapath or control role from a rule table.
4) *Anonymise and write the L0 files.* Nets, cells and port bits get names `n`k from one counter. Loki writes

`netlist.blif` and `netlist.v`, `graph.json` (nodes with features and labels, edges), `labels.csv` (one row per node), `words.json` (the word-level connectivity the scorer needs) and `key.json`. Only `key.json` holds original names; no analysis reads it.

5) *Verify.* Odin III's `equiv-check` compares the written BLIF with the floor's own netlist and the anonymised netlist with the original; both must read `equivalent` before the floor counts as built.
6) *L1: normalise.* `loki re` reads `netlist.blif` (never `key.json`) into a fresh Odin III design, level 0, in which every logic cell is a function (a LUT by its truth table, a gate by its library function). On `f1` with hard cells the `unmap` pass runs on the working copy and its facts feed the `hard` detector.
7) *L1: hypothesis rounds.* Each round runs the detectors, scores every hypothesis (Section III-C), proves the ones worth a proof, resolves conflicts and commits the confident ones (Algorithm 1); the next round sees the committed level.
8) *L1: commit and output.* A committed hypothesis is rebuilt as a word cell (`raise`) wrapped in a module (`submod`) tagged with its kind, confidence and recovered function. At the fixed point Loki writes `design.v`, `overlay.json` (every hypothesis with its evidence) and `report.json`, and checks the final design against the input with `equiv-check`.
9) *L2.* `loki analyze` reads `graph.json` only: structural features, four clusterings and a GNN (Section III-D).
10) *Score.* `loki score` compares the overlay with the answer key (Section III-E); `loki score-all` aggregates every run into per-kind tables.

In this micro the comparator cone of the `f3` floor has 26 logic cells (L1 spec, Section 3); the `f2` LUTs carry the comparator spans that the join reads back.

IV. The VerilogEval benchmark and its structure

A. VerilogEval spec-to-rtl as the Gate-Breaker benchmark

Gate-Breaker evaluates on VerilogEval [2], taking the spec-to-rtl variant: 156 problems, each a natural-language prompt, a reference solution (`*_ref.sv`, module `RefModule`) and a testbench (`*_test.sv`). The Gate-Breaker flow synthesizes the reference to a cmos6 gate netlist, hands the netlist to the model, and checks the model's RTL against the testbench. Loki uses the same 156 references as L0 sources and reads the reference module as the design; the prompts and testbenches are not inputs to L0, L1 or L2. The references are SystemVerilog, read through Odin III's slang adapter (Section V-A).

B. The complexity score and its quartiles

Gate-Breaker groups the problems by a complexity score (its Eq. 2), a weighted sum of counts of HDL constructs:

$$C = 1000\,c_{\mathrm{Seq}} + 200\,(c_{\mathrm{FG}} + c_{\mathrm{F}}) + 20\,c_{\mathrm{Mo}} + 15\,(c_{@A} + c_{\mathrm{B}}) + 10\,(c_{\mathrm{M}} + c_{\mathrm{Ca}}) + 5\,c_{\mathrm{E}} + 3\,c_{\mathrm{A}} + 2\,c_{\mathrm{C}}. \quad (3)$$

The paper names the terms but does not define them, so the implementation we reuse (`complexity.py` of the second Gate-Breaker replication) fixes the reading: c_{Seq} counts clocked `always` blocks (a `posedge` or `negedge` in the sensitivity list), c_{FG} for-generate constructs, c_{F} `for` loops, c_{Mo} module instantiations, $c_{@A}$ all `always` blocks, c_{B} declarations wider than one bit, c_{M} multiplexers (ternary operators plus non-default `case` items), c_{Ca} `case` statements, c_{E} arithmetic operators (`+ - * / %`), c_{A} continuous assignments and c_{C} concatenations. A clocked block weighs 1000 on its own, so a design in which the counter recognises one scores at least 1000, which is Q3 or Q4.

The quartiles are the paper's absolute bins (its Table II), not rank quartiles of the benchmark: Q1 is 3 to 23, Q2 24 to 111, Q3 112 to 1064 and Q4 1065 to 1326. The 156 references fall 38, 39, 40 and 39 into Q1 to Q4, close to the paper's 39 per quartile. The 150 designs that build on every floor are split 38, 39, 37 and 36: the six missing ones are `Prob031`, `Prob046` and `Prob145` (Q3) and `Prob078`, `Prob151` and `Prob156` (Q4). Table III gives the counts.

Deviation: pyverilog AST against a regex fallback. The score is computed on the pyverilog AST of the first module when the file parses, and by regular-expression counts of the same terms otherwise, with the method recorded per design. Because the references are SystemVerilog, pyverilog parses only 60 of the 156; the other 96 use the regex counts. The split is skewed by quartile (Table III): 36 of Q1 and 23 of Q2 are AST-scored, but 1 of Q3 and none of Q4. The regex counts operator characters and keywords textually, so they differ from an AST count (the observed Q4 range, 1069 to 3259, exceeds the paper's bin top of 1326 for this reason or because the problems differ; we do not separate the two). The 156 scores equal those of the replication that computed them first, exactly, so the quartiles here and in Table XII are the same partition.

C. The feature matrix

Gate-Breaker characterises its benchmark by the HDL structures it contains. We build the same kind of matrix for the 150 built designs, from the `f3` floor of each, with rules fixed in advance (`summarize.py`): *arithmetic* counts the word cells of type `$add $sub $mul $div $mod $neg $pos $pow $alu $macc`; *select/mux* those of type `$mux $pmux $bmux $demux`; *storage* the register words (`$dff`, `$dffe`, `$adff`, `$adffe`, `$sdff`, `$sdffe`, `$sdffce`, `$dffsr`, `$aldff`, `$dlatch`, `$adlatch`); *FSM* is 1 when Odin III's `fsm` pass marked any word; *control* is 1 when any label row has role `control` or `mixed`; and *wide buses* counts the distinct bus names with a bit index of at least 1. The matrix uses one symbol per cell: ✗ for count 0, ✓ for one, ✓✓ for two or more; FSM and control are 0/1 flags, so they show ✗ or ✓ only.

The bus column is `buses_wide`, not `buses`. The latter counts every named signal, including single-bit ports, so it is at least 1 for all 150 designs and at least 2 for 147, and would put a check mark in every row. The wide count is the quantity

TABLE III
THE VERILOGEVAL SPEC-TO-RTL REFERENCES BY GATE-BREAKER COMPLEXITY QUARTILE (ABSOLUTE BINS OF TABLE II; SCORE RANGE OBSERVED IN THE 156 REFS; PYVERILOG AST SCORES AGAINST REGEX-FALLBACK SCORES; BUILT = DESIGNS WITH ALL THREE FLOORS).

Quartile	Bin	Observed	Refs	AST/regex	Built
Q1	3–23	3–23	38	36/2	38
Q2	24–111	24–90	39	23/16	39
Q3	112–1064	120–1060	40	1/39	37
Q4	1065–1326	1069–3259	39	0/39	36
All			156	60/96	150

a "multi-bit bus" row means; both columns are committed in `features.csv`.

Table IV shows 12 designs, three per quartile at fixed ranks by size, so the sample is not hand-picked; Table V counts, per quartile, the designs that show each feature. The matrix shows what the weight of a clocked block (Eq. 3) leads one to expect: storage is absent from Q1 and Q2 (0 of 77 designs) and present in 31 of 37 Q3 and all 36 Q4 designs. The other features grow with the quartile: select/mux 8% in Q1 to 56% in Q4, control 3% to 83%, FSMs 0 of 77 below Q3 and 15 of 36 in Q4. Arithmetic is rare throughout (21 of 150, 14%), so the benchmark leans on control, selection and storage more than on arithmetic datapaths. Wide buses are common from Q2 up (90%, 59%, 100%) but nearly absent in Q1 (8%), whose members mostly have only single-bit signals. The FSM flag is Odin III's detection, not a reading of the source: `Prob107_fsm1s` in Table IV has storage and control cells but no FSM mark.

D. Benchmark size per technology mapping

Table VI gives the size of the 150 designs on each floor. The averages hide a heavy tail. Over all designs the mean node count is 163.5 on `f1-k6`, 111.4 on `f2` and 362.0 on `f3`, but the medians are 14.0, 13.5 and 25.0: most references are small (125 of 150 have at most 100 nodes at `f3`) and a few are large (9 exceed 1000; the largest, `Prob144_conwaylife`, has 15,487). The gate mapping roughly triples the node count of the LUT mapping (362.0 against 111.4 on average, 25.0 against 13.5 at the median), as expected when a LUT of up to six inputs becomes a network of cells from a library of one- and two-input gates (BUF, INV, NAND2, NOR2). `f1-k6` sits above `f2` on average and in Q3 and Q4 (109.4 against 77.4; 453.9 against 269.8) while the two coincide in Q1 and Q2 (6.3 and 6.1; 100.2 and 100.0): the `f1-k6` floor keeps hard cells (21 designs hold hard adders) where `f2` spreads arithmetic over LUTs, which is consistent with the excess but is not isolated here. By quartile on `f3`, the mean grows from 9.9 nodes in Q1 to 972.1 in Q4, but the Q4 median is only 51.5: the complexity score tracks the number and kind of HDL constructs, not the netlist size. For scale, the micro suite's `f3` floors hold 50,047 nodes over 65 designs, a mean of about 770.

TABLE IV
BENCHMARK FEATURE MATRIX, A SAMPLE OF 12 OF THE 150 DESIGNS (THE DESIGNS AT RANKS $\lfloor n/6 \rfloor$, $\lfloor n/2 \rfloor$ AND $\lfloor 5n/6 \rfloor$ BY F3 NODE COUNT INSIDE EACH QUARTILE). ✗: FEATURE ABSENT; ✓: ONE; ✓✓: MORE THAN ONE; FSM AND CONTROL ARE 0/1 FLAGS. FEATURES ARE READ FROM THE F3 FLOOR (SECTION IV-C).

Design	Q	f3 nodes	Arith	Sel	Stor	FSM	Ctl	Bus
Prob028_m2014_q4a	Q1	3	✗	✗	✗	✗	✗	✗
Prob036_ringer	Q1	8	✗	✗	✗	✗	✗	✗
Prob091_2012_q2b	Q1	17	✗	✗	✗	✗	✗	✓
Prob125_kmap3	Q2	15	✗	✗	✗	✗	✗	✗
Prob015_vector1	Q2	32	✗	✗	✗	✗	✗	✓✓
Prob123_bugs_addsubz	Q2	166	✓✓	✓	✗	✗	✗	✓✓
Prob107_fsm1s	Q3	11	✗	✓	✓	✗	✓	✗
Prob117_circuit9	Q3	28	✓	✓	✓✓	✗	✓	✓
Prob045_edgedetect2	Q3	89	✗	✗	✓✓	✗	✗	✓✓
Prob128_fsm_ps2	Q4	30	✗	✗	✓	✓	✓	✓✓
Prob140_fsm_hdlc	Q4	52	✗	✗	✓	✓	✓	✓
Prob115_shift18	Q4	1121	✗	✓✓	✓✓	✗	✓	✓✓

V. RESULTS

A. L0 on VerilogEval: the provenance ceiling

The provenance experiment is run on the 156 VerilogEval spec-to-rtl reference designs (Section IV) next to the micro suite. The question is the one L0 exists to answer: how much of the reference does the answer key reconstruct, per floor, with no recovery step at all. The experiment builds the three floors `f1-k6`, `f2` and `f3` for every reference (`f1-earch` is not built; one architecture is enough because `f2` and `f3` do not depend on it).

Coverage of the build. Of the 156 references, 150 build on all three floors, and `loki check` reports zero errors on the assembled dataset. Two are skipped by Odin III's SystemVerilog reader: `Prob151_review2015_fsm` and `Prob156_review2015_fancytimer` (both Q4), with `ODIN3_ERR_PARSE ... an explicit 4-bit cast is not supported`. Four fail Loki's own equivalence gate and are recorded as failed rather than forced in: `Prob031_dff` (the reference initialises `q` to `1'hx`; the writer-versus-Odin III BLIF check reports `differs` on `f1` and `f2`, while `f3` was equivalent) and `Prob046_dff8p`, `Prob078_dualedge` and `Prob145_circuit8`, whose negedge flip-flops make `equiv-check` return an error status for both comparisons. One design, `Prob144_conwaylife`, was killed by Wyrd's default 2 GB step memory limit (peak 4.3 GB) and built after a rerun with an 8 GB limit. Reading `.sv` sources needed two fixes in Loki (SystemVerilog is read with `--nolatches` only, since `--vtr-compat` is refused for it, and the slang adapter is registered with the process); both are in `loki/odin.py` and apply to the experiment run.

The ceiling. On `f3` the 150 designs have 54,306 gate nodes. Every node is labelled, cell-typed and carries an HDL span: the label, `cell_type` and HDL coverage have geometric mean 1.000 over designs. The 150 module scopes of the 150 reference modules are all present (module coverage 1.000). Signal coverage, the recovered leaf signal names over the signals the reference declares, has geometric mean 0.986

TABLE V

DESIGNS SHOWING EACH FEATURE, PER COMPLEXITY QUARTILE: COUNT (PERCENT OF THE QUARTILE). THE LAST COLUMN COUNTS DESIGNS WITH TWO OR MORE INSTANCES OF THE FEATURE (ALWAYS 0 FOR THE 0/1 FLAGS). WIDE BUSES ARE DISTINCT MULTI-BIT BUSES; A DESIGN WITH ONLY SINGLE-BIT PORTS HAS NONE.

Feature	Q1 (n=38)	Q2 (n=39)	Q3 (n=37)	Q4 (n=36)	All (n=150)	≥ 2
Arithmetic	2 (5%)	4 (10%)	7 (19%)	8 (22%)	21 (14%)	9
Select/mux	3 (8%)	10 (26%)	12 (32%)	20 (56%)	45 (30%)	12
Storage	0 (0%)	0 (0%)	31 (84%)	36 (100%)	67 (45%)	12
FSM	0 (0%)	0 (0%)	1 (3%)	15 (42%)	16 (11%)	0
Control	1 (3%)	4 (10%)	15 (41%)	30 (83%)	50 (33%)	0
Wide buses	3 (8%)	35 (90%)	22 (59%)	36 (100%)	96 (64%)	55

TABLE VI

BENCHMARK SIZE PER TECHNOLOGY-MAPPING FLOOR, 150 DESIGNS: MEAN / MEDIAN NODE COUNT (TOP), AND PER COMPLEXITY QUARTILE (BOTTOM; NODES). NODES ARE ALL GRAPH NODES OF THE FLOOR NETLIST: PRIMARY INPUTS AND OUTPUTS, CONSTANTS, CELLS.

All 150 designs	f1-k6	f2	f3
Nodes	163.5 / 14.0	111.4 / 13.5	362.0 / 25.0
Cells	114.6 / 5.0	62.5 / 4.0	313.2 / 13.5
Nets	218.3 / 12.0	93.5 / 10.0	344.2 / 21.0
Nodes by quartile	**f1-k6**	**f2**	**f3**
Q1 (n=38)	6.3 / 5.0	6.1 / 5.0	9.9 / 8.0
Q2 (n=39)	100.2 / 26.0	100.0 / 26.0	238.2 / 32.0
Q3 (n=37)	109.4 / 18.0	77.4 / 13.0	260.6 / 28.0
Q4 (n=36)	453.9 / 26.5	269.8 / 26.5	972.1 / 51.5

TABLE VII

THE L0 CEILING ON `f3`, MICRO SUITE AND VERILOGEVAL REFERENCES. BOTH COLUMNS ARE COUNTS OVER ALL BUILT DESIGNS; THE LAST ROW IS THE PER-DESIGN COVERAGE OF DECLARED SIGNALS.

	Micro	VerilogEval
Source designs	68	156
Built on `f3`	65	150
Refused by the reader	3	2
Failed equivalence gate	0	4
Gate nodes, all labelled	50,047	54,306
Rows with `cell_type` and HDL span	100%	100%
Module scopes / reference modules	112 / 113	150 / 150
Leaf signal names / declared signals	611 / 666	841 / 745
Per-design signal coverage: median	100%	100%
minimum	55.6%	75.0%
geometric mean	–	0.986

(arithmetic mean 1.029): 120 designs are at exactly 1, 28 are below it, and 2 are above it, where flattened array elements add names. The micro suite shows the same shape with a slightly lower ceiling (Table VII): 99.1% of module scopes and 91.7% of signal names (an aggregate ratio, where VerilogEval's 841 over 745 is inflated by those two designs; capped at each design's declared count it is 715 over 745, 96.0%). The two sets agree on the message of Section V-C: labels and spans are complete, and what the mapping erased is the signal names, not the labels.

Caveats, reported with the numbers. (i) `--nolatches` turns inferred-latch paths into don't-care and `lower --async sync` turns asynchronous resets into synchronous clears, so a built netlist can differ from the reference's behaviour even though Loki's own checks (netlist against its anonymised twin and against Odin III's writer) pass; the build gate guarantees internal consistency, not equality with the reference. (ii) The reference module, instance and signal counts come from pyverilog where it parses the reference (130 of the 150) and from a regex fallback otherwise (20); the regex counts are approximate and the reference signal count is the weakest number in this section. (iii) The same pyverilog limit decides the complexity quartiles (Section IV-B). (iv) A train/test split by design is fixed by the manifest rule (117 train, 33 test on `f3`); it is not used here, where no model is fitted.

B. L1 recovery: what the flow already reverse engineered, and what it erased

Table VIII reports L1 on the renamed floors (identical to base, as the neutrality check requires). The head-line result is the `hard` detector: hard adders recover 67 of 67 with precision 0.985, hard multipliers 30 of 30, because the synthesis flow itself already reverse engineered them and `unmap` says so. The rest is the honest difficulty: on f1 the calibrated macro-F1 is 0.354 (counted 271, matched 107, ground truth 688), with `ff_en` at 0.364, registers at 0.021 and muxes at 0.018; on f2 it is 0.108 (91/19/686), with adders at F1 0.237 (precision 1.000, recall 0.134) and most other kinds near zero. Without calibration the flow proposes far more and confirms little: f1 macro-F1 0.202 with 1,128 counted (323 of them declined-adder proposals), f2 0.075. Calibration is honest about itself: ECE 0.217 calibrated against 0.650 uncalibrated.

C. f3: recovery from the cmos6 gate netlist

Gate-Breaker's own input is a cmos6 gate netlist, so we built the f3 floor (65 of 68 micro designs; the gate reader refuses three hard-block sources) and asked the same questions of it. The L0 answer key is complete at this level: all 50,047 gate nodes carry a label row, a cmos6 `cell_type` and an HDL span. What provenance reconstructs of the reference is nearly complete but not total: 112 module scopes against 113 module definitions (99.1%; 54 of 65 designs complete) and 611 distinct leaf signal names against 666 declared signals (91.7%; median design 100%, minimum 55.6%). The module gap is reference-side (generate branches and unused submodules never elaborate); the signal gap is synthesis-side (nets that mapping created or merged have no original name left). The

TABLE VIII

L1 RECOVERY ON THE RENAMED FLOORS, PER KIND. PRECISION/RECALL/F1 AGAINST THE L0 GROUND TRUTH; CALIBRATED (MICRO-FIT) COLUMNS. THE HARD KINDS RECOVER FROM `UNMAP` FACTS; THE REST MUST BE RE-FOUND FROM THE GATES.

Kind (f1)	Precision	Recall	F1
hard_add	0.985	1.000	0.993
hard_mul	1.000	1.000	1.000
ff_en	–	–	0.364
register	–	–	0.021
mux	–	–	0.018
mux_bit	–	–	0.081
macro-F1 (all kinds)			0.354
Kind (f2)			
adder	1.000	0.134	0.237
subtractor	1.000	0.074	–
mux_bit	–	–	0.100
adder_cin	–	–	0.029
macro-F1 (all kinds)			0.108

ceiling is consequently what the mapping preserved, not the source RTL.

Table IX reports L1 on the 15 f3 test designs (23,098 gate nodes), base and renamed, with a calibration fitted on the 50 train designs' f3 floors. The result is the strict one: no construct is counted (0 of 144 ground-truth instances; macro-F1 0.0) and no hypothesis is committed at all. The f3 priors are strongly negative because the gate-level detectors are imprecise on the train floors (comparator -9.24 from 31,036 proposals with 2 true), so the best calibrated confidence in any test design is a bus at $p = 0.764$, below the 0.9 commit threshold, and the L1-stop Verilog is byte-identical to the input netlist in 15 of 15 designs. Without calibration the detectors count 1,166 constructs for 2 matches (macro-F1 0.038; one adder, one register) and 2,835 buses for 0 matches. Buses are proposed but never reconstructed: 2,569 counted bus hypotheses against 120 ground-truth buses, 0 matched, so no width or bit order is recovered. The role detector labels every logic cell datapath, which is 100% correct only because the f3 answer key holds no control cells (0 of 50,047; f1 has 4): control-versus-data is not answerable from f3 provenance, and the datapath F1 of 1.000 over 20,541 cells is vacuous. The hard detector stays f1-only by design (`unmap` runs only on f1 floors with hard cells), so the 28 hard ground-truth instances are unreachable here. Base and renamed `score.json` digests are identical (9368cfc26303): naming contributes no decision bit at f3 either.

The AST and CFG deltas (Table X) follow from the zero-commit result: with the f3 calibration L1-stop is byte-identical to L0-stop, so both metrics have delta exactly 0.0000 in every quartile (AST over 15 designs; CFG over the 13 whose CFG the probe can finish, 12,000 nodes being its practical $O(n^2)$ limit). The uncalibrated diagnostic, where L1 commits 1,535 hypotheses in 11 of 15 designs, moves the means by +0.0111 (AST) and -0.0075 (CFG), with mixed per-quartile signs; the commits are mostly wrong `eq_const` and bus pieces, so this is not evidence about recovery quality. A flatten control (Odin III `proc; flatten`, which also lowers the committed processes) inlines the committed hierarchy and raises both metrics over the gate netlist: AST 0.4576 against 0.4044 (+0.0533, all 15 designs) and CFG 0.6941 against 0.6382 (+0.0559, the 12 designs the probe can score). Gate-Breaker's Algorithms 2 and 3 read a single module, while Loki's L1 commits a hierarchy, so a recovery that produces hierarchy is systematically penalised until it is flattened: for a netlist-to-RTL benchmark this is a metric limitation, not a property of the recovery.

TABLE IX

F3 L1 PER KIND ON THE 15 TEST DESIGNS. THE CALIBRATED COLUMNS ARE THE PROTOCOL'S SETTING; THE `--CALIB NONE` COLUMNS ARE A DIAGNOSTIC OF WHAT THE DETECTORS PROPOSE BEFORE CALIBRATION PRUNES IT. A KIND WITH NO COUNTED PREDICTION HAS NO F1 AND IS LEFT OUT OF THE MACRO AVERAGE.

		calibrated			`--calib none`			
Kind	GT	counted	matched	F1	counted	matched	precision	F1
adder	17	0	0	–	1	1	1.000	0.111
adder_cin	17	0	0	–	0	0	–	–
comparator	20	0	0	–	14	0	0.000	0.000
eq_const	16	0	0	–	1,141	0	0.000	0.000
ff_srst	2	0	0	–	7	0	0.000	0.000
hard_add	17	0	0	–	0	0	–	–
hard_mul	11	0	0	–	0	0	–	–
mux	7	0	0	–	0	0	–	–
mux_bit	7	0	0	–	0	0	–	–
register	22	0	0	–	3	1	0.333	0.080
subtractor	8	0	0	–	0	0	–	–
macro-F1	144	0	0	0.000	1,166	2		0.038
buses	120	2,569	0	0.000	2,835	0	0.000	0.000
datapath role	20,541	20,541	20,541	1.000	20,541	20,541	1.000	1.000
control role	0	0	0	–	0	0	–	–

TABLE X

F3 AST (GATE-BREAKER ALGORITHM 2, DEPTH 3, PYVERILOG) AND CFG (ALGORITHM 3, ODIN3 PROBE) SIMILARITY AGAINST THE MICRO REFERENCE RTL, L0-STOP (GATE NETLIST) AGAINST L1-STOP (`DESIGN.V`); QUARTILES BY F3 NODE COUNT. WITH THE F3 CALIBRATION L1-STOP EQUALS L0-STOP, SO EVERY DELTA IS 0.0000; THE TABLE THEREFORE SHOWS THE L0 VALUES AND THE DIAGNOSTIC (`--CALIB NONE`) DELTAS.

Quartile	AST L0	AST L1	ΔAST	CFG L0	CFG L1	ΔCFG
0 (4 designs)	0.4428	0.4428	0.0000	0.7909	0.7909	0.0000
1 (4)	0.3765	0.3765	0.0000	0.4224	0.4224	0.0000
2 (4)	0.3969	0.3969	0.0000	0.7013	0.7013	0.0000
3 (3 AST / 1 CFG)	0.4003	0.4003	0.0000	0.7507	0.7507	0.0000
mean (15 / 13)	0.4044	0.4044	0.0000	0.6468	0.6468	0.0000
diagnostic `--calib none` (L1 commits hierarchy)						
0 (4)	0.4428	0.4277	−0.0151	0.7909	0.7776	−0.0133
1 (4)	0.3765	0.3756	−0.0009	0.4224	0.4099	−0.0124
2 (4)	0.3969	0.4542	+0.0573	0.7013	0.7014	+0.0001
3 (3 AST / 1 CFG)	0.4003	0.4007	+0.0004	0.7507	0.7561	+0.0055
mean	0.4044	0.4155	+0.0111	0.6468	0.6394	−0.0075

D. L2: clusters that carry the hierarchy, and a GNN that beats 1-WL

Table XI scores the four clusterings against the known module hierarchy. Louvain leads (modularity 0.4220, NMI 0.1360); k-means and affinity propagation do not find the hierarchy (negative modularity), which is itself a result: graph structure alone, without the labels, favours the community methods. 271 of 272 partition records are identical between

TABLE XI
L2 CLUSTERING AGAINST THE KNOWN MODULE HIERARCHY (68 DESIGNS, MEAN OVER THE RENAMED FLOORS, IDENTICAL TO BASE). MODULARITY AND NMI(MODULE_PATH) PER METHOD.

Method	Modularity	NMI
Louvain	0.4220	0.1360
Spectral	0.3620	0.1378
Affinity propagation	-0.2264	0.2708
k-means	-0.2912	0.3220

TABLE XII
GATE-BREAKER METRICS ON THE VERILOGEVAL QUARTILES (ONE RUN, K=10, 4 WORKERS, 180 S PER CALL, 7,440 S WALL CLOCK, 740 CALLS; DELTAS VS THE PAPER'S O3-MINI IN THE TEXT).

Quartile (n)	SR	S@1	S@5	S@10	AST	CFG
Q1 (36)	0.903	0.944	1.000	1.000	0.894	0.942
Q2 (37)	0.778	0.811	1.000	1.000	0.761	0.890

base and renamed floors; the one difference exposed a determinism defect in the spectral path (repeated runs on the same base design give modularities -0.0079, -0.0410 and 0.0252), reported as an open issue rather than hidden.

The GNN result answers the paper's question directly. A node-only GraphSAGE loses to the 1-WL baseline on real micros (macro-F1 0.2413 against 0.2711); combining node, graphlet, WL and NPN features turns it around (0.3545 against 0.2711 on the earlier real-micro run), and the full labelled run of this experiment gives test macro-F1 0.4241 against 0.2717 (accuracy 0.8863 against 0.8492, 3,932 test nodes across 13 designs). The features carry the recovery, not the message passing alone.

E. The Gate-Breaker metrics

Gate-Breaker's metrics compare a recovered RTL against a reference: Success Rate, Success@k, AST similarity and CFG similarity. Loki recovers structure, not a new RTL text, so the direct comparison is the replication: the Gate-Breaker harness itself, run on the simple benchmarks with the only language model available on the host, gave success rate 0.933 on Q1 and 0.829 on Q2, Success@10 1.000 in both quartiles, AST similarity 0.576 and 0.444, CFG similarity 1.000 and 0.736, with 169 of 174 extracted candidates functionally correct – a first replication on the fallback micro set, not the paper's VerilogEval set. The VerilogEval replication, on the quartiles the paper used, is reported in Table XII with the AST and CFG kernels recomputed with common-width feature vectors (the first run padded per graph and truncated in the zip, which the v1.1.0 CFG probe found; those columns are not comparable to it).

Seventy-four of the 78 Q1+Q2 designs are usable (three excluded because their synthesized netlists fail the official testbench self-check, one because its 241 KB prompt exceeds the OS argument limit); every scored design is solved by attempt 5. The deltas against the paper's O3-Mini: Q1 success rate -0.092, AST -0.030, CFG -0.011; Q2 success rate -0.039, Success@5 +0.125, Success@10 +0.111, AST -0.164, CFG +0.083. The Q1 gap is harness no-output, not wrong RTL; the Q2 AST gap is truth-table and case references against the model's boolean expressions. Two caveats: the host's only model path routes through a gateway that aliases a DeepSeek backend as Claude Opus (so the model column is not verifiably the paper's comparison class), and the CLI exposes none of the paper's sampling flags (T=0.85, p=0.95). The CFG column uses the common-width node kernel (the odin3 CFG probe); the AST column uses the paper's pyverilog Algorithm 2 of Gate-Breaker.

What the two lines of work measure is complementary. The language model recovers functional behaviour from the gates and names nothing reliably; Loki recovers the structures the flow preserved (hard blocks, the hierarchy, the signal roles its features see) and proves what it claims. The Gate-Breaker future work – bus naming, datapath/control classification, partitioning – is exactly L2's table, and its numbers are the baseline a model-assisted Loki would have to beat.

VI. DISCUSSION

Three observations follow. First, *the answer depends on what the flow erased*. The structures that survived synthesis recover almost perfectly without any intelligence beyond reading the pass's own facts; the structures synthesis erased (the counter behind its flip-flop, the mux behind its gates) recover weakly, and no amount of honest Bayesian machinery changes that without better detectors or a model. Second, *the graph features carry the recovery*: the GNN only beats 1-WL when node, graphlet, WL and NPN features combine, which is the same lesson as the clustering table (community methods find the hierarchy, centroid methods do not). Third, *the benchmarks made the limits visible*: the renamed-instance protocol proved naming neutrality over 105,646 identifiers, and the same protocol exposed a rename-script defect (library `.blackbox` model names were renamed, which silently removed every hard proposal on renamed floors) and the spectral determinism defect – both fixed or reported, and both invisible without the protocol.

VII. CONCLUSION

Loki III asks how far netlist-to-RTL reverse engineering gets without a language model. On the benchmarks of this paper the answer is: the already-reverse-engineered structures, perfectly; the rest, weakly but measurably, with calibration that reports its own error, a graph stage whose features beat 1-WL, and clusters that carry the designer's hierarchy. The methodology – provenance-tracked labelled datasets plus the renamed-instance protocol – is the part we would export to any future comparison, with or without a model.

REFERENCES

[1] M. O. Faruque, P. Jamieson, A. Patooghy, and A.-H. A. Badawy, "Gate-breaker: An LLM-powered netlist-to-RTL reverse engineering tool," in *Proc. IEEE Int. Performance, Computing, and Communications Conf. (IPCCC)*, 2025, pp. 363–370.

[2] M. Liu, N. Pinckney, B. Khailany, and H. Ren, "VerilogEval: Evaluating large language models for Verilog code generation," in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2023.
[3] F. Gharibian, L. Shannon, and P. Jamieson, "Finding system-level information and analyzing its correlation to FPGA placement," in *Proc. Int. Conf. Field Programmable Logic and Applications (FPL)*, 2010, pp. 544–549.
[4] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *Int. Conf. Learning Representations (ICLR)*, 2019.
[5] L. Alrahis, A. Sengupta, J. Knechtel, S. Patnaik, H. Saleh, B. Mohammad, M. Al-Qutayri, and O. Sinanoglu, "GNN-RE: Graph neural networks for reverse engineering of gate-level netlists," *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 2022.